Java Interview Questions

Complete Reference — All Questions from Notes

Core Java • OOP • Java 8 Features • Collections • Multithreading
Exception Handling • Hibernate/JPA • Spring Boot • REST API • Experience Q&A

1. Core Java Fundamentals

Q: Why is Java not 100% object-oriented?

It supports primitive data types like int, byte, long, short, etc., which are not objects. Hence these data types like int, float, double, etc., are not object-oriented. That's why Java is not 100% object-oriented.

Q: Why are pointers not used in Java?

• **Memory Safety:** Java avoids pointers to prevent direct memory access, reducing the risk of memory corruption.

• **Simplified Syntax:** By not using pointers, Java simplifies programming and reduces complexity.

• **Automatic Garbage Collection:** Java manages memory automatically, eliminating the need for explicit pointer management.

Q: What is the JIT Compiler in Java? Why to use?

The JIT Compiler in Java converts bytecode to native machine code at runtime, speeding up execution and improving performance.

Q: What is the JVM and why is it important?

The JVM (Java Virtual Machine) is a virtual machine that runs Java bytecode, providing an environment for Java programs to execute on any device or operating system. It allows Java to be platform-independent by converting the compiled Java code (bytecode) into machine-specific code at runtime.

Q: What is a ClassLoader in Java? What are the different types?

A ClassLoader is responsible for loading classes into the Java Virtual Machine (JVM) at runtime. It dynamically loads classes based on the class path and helps in managing class dependencies and ensuring that class files are available when needed.

Types: Bootstrap ClassLoader, Extension ClassLoader, System/Application ClassLoader, Custom ClassLoader.

Q: Why is String immutable in Java?

Strings in Java are immutable to ensure thread safety, optimize memory usage, and improve security by preventing unexpected changes to string values. The String pool requires strings to be immutable; otherwise shared references can be changed from anywhere.

Q: What is the difference between String, StringBuilder, and StringBuffer?

• **Storage:** String uses String pool; StringBuilder and StringBuffer use heap memory.

• **Mutability:** String is immutable; both StringBuilder and StringBuffer are mutable.

• **Efficiency:** String is slow. StringBuilder is fastest. StringBuffer is faster than String but slower than StringBuilder.

• **Thread-safe:** StringBuffer is thread-safe (suitable for multiple threads). StringBuilder is NOT thread-safe (single thread only).

```
String first = "InterviewBit"; StringBuffer third = new
StringBuffer("InterviewBit"); StringBuilder fourth = new
StringBuilder("InterviewBit");
```

Q: What is the difference between == and .equals()?

== compares if two references point to the exact same memory location, while .equals() compares the actual contents or values of two objects. For objects like String, .equals() is typically used to check if the values are the same, not the memory references.

Q: What is the superclass of all Java classes?

java.lang.Object. Every class in Java either directly or indirectly inherits from Object, making it the root of the class hierarchy.

Some commonly used methods from Object include: toString(), equals(Object obj), hashCode(), getClass().

Q: What methods does the Object class have?

• toString() — Returns a string representation of the object.

• equals(Object obj) — Compares this object with the specified object for equality.

• hashCode() — Returns a hash code value for the object.

• getClass() — Returns the runtime class of the object.

• clone() — Creates and returns a copy of the object (protected; requires implementing Cloneable).

• notify() — Wakes up a single thread waiting on this object's monitor.

• notifyAll() — Wakes up all threads waiting on this object's monitor.

• wait() — Causes the current thread to wait until another thread invokes notify() or notifyAll().

• wait(long timeout) — Waits for the specified time or until notified.

• wait(long timeout, int nanos) — Waits for the specified time plus nanoseconds or until notified.

Q: What is pass by value and pass by reference?

• **Pass by Value:** Copy of the value is passed. Changes to parameters do not affect the original argument (Java, for primitives).

• **Pass by Reference:** Reference of the original variable is passed. Changes to parameters affect the original argument (C++, when using references).

Q: What are wrapper classes in Java?

Wrapper classes in Java are used to convert primitive data types into objects. Each primitive type (like int, char, double, etc.) has a corresponding wrapper class (Integer, Character, Double, etc.). They are part of the java.lang package. This is useful because Java collections can only store objects, not primitive types.

```
int num = 5; Integer wrappedNum = Integer.valueOf(num); // Manual boxing int originalNum = wrappedNum.intValue(); // Unboxing
```

Q: What is Serialization and Deserialization?

Serialization: The process of converting an object into a byte stream. This byte stream can then be saved to a file, sent over a network, or stored in a database. Ex: read file and save file to DB.

Deserialization: The reverse process, where a byte stream is converted back into a Java object. Ex: read file from DB and show data to user.

Q: What does the transient keyword mean in Java?

transient is a variable modifier used in serialization. At the time of serialization, if we don't want to save the value of a particular variable in a file, then we use the transient keyword.

Q: Why is char[] preferred over String for passwords?

char[] is mutable, meaning its contents can be modified (e.g., cleared) after use. On the other hand, String is immutable, meaning once it's created, it cannot be changed.

Q: What is the difference between Java SE and Java EE?

• **Java SE:** Provides the core Java platform for developing general-purpose applications, including basic APIs like java.lang, java.util, java.sql, etc. Suitable for desktop, mobile, and standalone applications.

• **Java EE:** Builds on Java SE and adds APIs for building large-scale, distributed, and enterprise-level applications, such as web services, servlets, JSPs, EJBs, and more. Tailored for server-side and cloud-based applications.

Q: What is a JAR and WAR file?

JAR (Java Archive): A zip-like file containing Java class files, libraries, resources, and metadata. Used to package and distribute Java applications or libraries.

WAR (Web Application ARchive): Used to package web applications (Java Servlets, JSPs, HTML, CSS, JS). WAR files are deployed on web servers like Apache Tomcat.

Q: What is the main objective of Garbage Collection?

The main objective is to free up the memory space occupied by unnecessary and unreachable objects during Java program execution. This ensures that memory resources are used efficiently, but it provides no guarantee that there would be sufficient memory for

program execution.

Q: What is the name of the Garbage Collector method?
finalize()

Q: What is the keyword to define a constant in Java?
final

Q: What happens if a constructor is declared private?
The class cannot be instantiated outside. This is used in the Singleton pattern.

Q: Can a try block exist without a catch block?
Yes, a try block can exist with only a finally block (try-finally), without a catch block.

Q: Can interface extend another interface?
Yes, an interface can extend another interface.

Q: What is the use of application.properties file in Java?
To configure various settings. It allows developers to define environment-specific properties such as database configurations, server settings, and application-specific settings in a centralized location. Also used to manage environment-specific configurations using profiles.

Q: What is the difference between a while loop and a do-while loop?
A while loop executes the loop body if the condition is true, while a do-while loop executes the loop body at least once before checking the condition.

Q: What is the Front Controller pattern in Java?
The Front Controller pattern means that all incoming HTTP requests are handled by a single controller which then dispatches the request to the appropriate handler. In the new way, we have multiple controllers but write a single controller and give paths — if paths are equal it calls that method's functionality. Think of it like a receptionist at an office — everyone comes through them first and then they direct you to the right department.

Q: How many ways can you create an object in Java?
- Using the **new** keyword (most common) — MyClass obj = new MyClass();
- Using **Reflection** — Class.forName("MyClass").getDeclaredConstructor().newInstance();
- Using **clone()** method — obj2 = (MyClass) obj1.clone();
- Using **Factory Methods / Design Patterns** — MyClassFactory.createInstance();
- Using **Deserialization** — ObjectInputStream.readObject();

Q: What is the difference between static and non-static methods?
Static method: Belongs to the class and can be called without creating an instance or object of that class.

Non-static method: Requires creating an instance/object of the class to call that method.

Q: What is Swagger?

Swagger is an open-source framework that simplifies the process of designing, building, documenting, and consuming RESTful web services. It provides tools for automatically generating interactive API documentation from your code.
Common annotations: @Api, @ApiOperation, @ApiParam, @ApiResponses, @ApiResponse.

Q: What is OpenAPI?

OpenAPI (formerly known as Swagger) is a specification for building APIs. It defines a standard way to describe REST APIs, including: Endpoints and their operations (GET, POST, PUT, DELETE), Input parameters and output responses, Data models (schemas), Authentication methods, Error messages and response codes.

Q: What is REST and RESTful?

REST represents Representational State Transfer. RESTful web services are web services that follow the REST architectural concept. It is an architectural style for developing applications that can be accessed over the network.

Q: Why are HTTP methods stateless in REST?

HTTP methods in REST are stateless because the server doesn't remember previous requests.

Q: What are the REST API idempotent HTTP methods?

REST API idempotent HTTP methods are: GET, PUT, DELETE.

2. OOP Concepts

Q: What are the four pillars of OOP? (Summary)

- **Encapsulation:** Hides the internal details of an object and only exposes what is necessary.
- **Inheritance:** Reusing code from a parent class.
- **Polymorphism:** 'Many forms' — methods can perform differently based on the object. Allows method overriding (runtime) and method overloading (compile-time).
- **Abstraction:** Hiding complex details and showing only essential features.

```
// Inheritance class Vehicle { void start() { System.out.println("Vehicle started"); } } class Car extends Vehicle { void drive() { System.out.println("Car is driving"); } } // Polymorphism class Vehicle { void sound() { System.out.println("Vehicle sound"); } } class Bike extends Vehicle { void sound() { System.out.println("Bike sound"); } } // Encapsulation public class Account { private double balance; public double getBalance() { return balance; } public void deposit(double amount) { if (amount > 0) balance += amount; } } // Abstraction abstract class Shape { abstract void draw(); } class Circle extends Shape { void draw() { System.out.println("Drawing circle"); } }
```

Q: What is the difference between an abstract class and an interface?

Abstract class: Has both abstract and non-abstract methods. Cannot be instantiated. Extended using the extends keyword.

Interface: Contains only abstract methods (before Java 8). Implemented using the implements keyword. A class can implement multiple interfaces but extend only one abstract class.

```
public abstract class Shape { public abstract void draw(); } public interface Details { void draw(); void jump(); }
```

Q: Can one class extend two abstract classes at once?

No. It will face the Diamond Problem — the compiler won't know which method to call.

Q: What is a marker interface?

An interface having no data member and member functions — in simple terms, an empty interface is called a marker interface. Examples: Serializable, Cloneable.

```
public interface Cloneable { }
```

Q: Can you override a private or static method in Java?

No. Private methods are not visible to subclasses, so they cannot be overridden. Static methods are bound to the class rather than instances — they cannot be overridden in the traditional sense.

Q: What is a Singleton class and how to create one?

A Singleton class is a class whose only one instance can be created at any given time, in one JVM.

```
public class Singleton { private static Singleton instance; private Singleton() {} public static Singleton getInstance() { if (instance == null) { instance = new Singleton(); } return instance; } }
```

Q: Thread-safe Singleton

It uses double-checked locking to ensure a single instance across threads efficiently.

```
public class Singleton { private static volatile Singleton instance; private Singleton() {} public static Singleton getInstance() { if (instance == null) { synchronized (Singleton.class) { if (instance == null) instance = new Singleton(); } } return instance; } }
```

Q: What are the different ways to break a Singleton pattern?

Reflection: Using reflection, you can access the private constructor and create a new instance.

Serialization: When a singleton class is serialized and then deserialized, it can create a new instance.

Cloning: If the singleton class implements Cloneable, the clone() method can create a new instance.

Q: How can you make a class immutable?

• Declare the class as final to prevent subclassing.
• Make all fields private and final to ensure they are only set once and not accessible outside the class.
• Initialize all fields via a constructor and ensure there are no setter methods.
• Avoid methods that modify the object's state; provide getters for read-only access.
• If the class has mutable fields, return copies (e.g., for collections) to prevent external modification.

Q: What is Association, Aggregation, and Composition (HAS-A)?

Association: The relation between two classes. Has 2 types: Aggregation and Composition.
Aggregation (loosely coupled / HAS-A): One class contains objects of another class. No dependency — they can exist independently. Ex: Customer and Orders — even if the Customer account is deleted, Orders can still exist in the database.
Composition (tightly coupled): Represents a strong IS-A relationship where the child cannot exist independently. Ex: Car and Engine — if the Car is destroyed, the Engine is also destroyed.

Q: Why are Comparable and Comparator needed in Java?

Comparable: Allows a class to define its natural ordering. Implement Comparable<T> and override compareTo(T o). Sorting logic is in the class itself.
Comparator: Allows for custom ordering of objects without modifying the class. Implement Comparator<T> and override compare(T o1, T o2). Sorting logic is in a separate class.

Q: What are the pre-request steps for object call in Map?

Equals and hashCode methods must be properly overridden. Comparable and Comparator methods.

Q: What are design patterns in JDK?

Singleton pattern, Factory method, Builder pattern, Decorator pattern (used for logging).

Q: What is SOLID principles?

S — Single Responsibility: one job/purpose per class.
O — Open Closed: interfaces and abstract classes to extend without modifying.
L — Liskov Substitution: child class extends parent and can replace it without breaking.
I — Interface Segregation: don't force a class to implement unnecessary methods, split into smaller interfaces.
D — Dependency Inversion: depend on abstraction, not concrete classes.

Q: How many ways can we bind an object?

Static Binding (Early Binding): Binding happens at compile time. Applicable to static, final, or private methods and variables. The reference type determines the method or variable to bind.
Dynamic Binding (Late Binding): Binding happens at runtime. Applicable to non-static instance methods. The actual object type determines the method to bind, enabling

polymorphism.

Q: What is the DispatcherServlet response flow?

DispatcherServlet acts as the Front Controller — it receives every request and coordinates the entire response process.

1. **Request comes in** — Browser sends request → goes to DispatcherServlet.
2. **Handler Mapping** — DispatcherServlet asks HandlerMapping → which controller handles this URL?
3. **Controller executes** — Controller runs the business logic → returns a ModelAndView.
4. **View Resolver** — DispatcherServlet asks ViewResolver → which actual file matches this view name? Ex: 'home' → /WEB-INF/views/home.jsp
5. **View renders response** — View (JSP/Thymeleaf) uses the Model data → builds the HTML response.
6. **Response sent back** — DispatcherServlet sends the final response back to the browser.

Key point: DispatcherServlet doesn't know the response directly — it delegates to HandlerMapping, Controller, and ViewResolver. It just coordinates everything.

3. Java 8 Features

Q: What are the Java 8 features?

Lambda Expressions, Stream API, Default methods in Interface, Static methods in interfaces, Functional Interface, Method References, Date API (java.time), Optional class.

Q: What are the advantages of Java 8?

Compact Code (Less Boiler Plate Code), More readable and reusable code, More testable code, Parallel operations.

Q: What is a Lambda Expression?

A lambda expression is an anonymous function with no name, return type and access modifier having one lambda (->) arrow symbol.

```
// Regular method public int add(int a, int b) { int c = a+b; return c; } // Lambda using BiConsumer BiConsumer<Integer, Integer> bi = (a,b) -> System.out.println(a+b); bi.accept(10, 20);
```

Q: What is a Functional Interface?

An interface with only one abstract method is known as a functional interface. It can have any number of default or static methods. Examples: Comparable, Runnable.

```
@FunctionalInterface public interface MyFunction { void execute(); default void log() { System.out.println("Logging"); } static void staticMethod() { System.out.println("Static method"); } }
```

Q: How are lambda expressions and functional interfaces related?

Functional Interface is used to provide reference to lambda expressions.

```
Comparator<String> c = (s1,s2) -> s1.compareTo(s2);
```

Q: Can you create your own functional interface?

Yes. Steps:

1. Create an Interface.
2. Annotate it with @FunctionalInterface.
3. Define exactly one abstract method.

Q: Which functional interface is used for filtering data?

Predicate<T> — It takes a value and returns a boolean.

```
List<String> names = List.of("John", "Ava", "Steve"); names.stream().filter(name
-> name.startsWith("A")).forEach(System.out::println);
```

Q: Which interface is used in forEach()?

Consumer<T> — It consumes the value without returning anything.

```
List<Integer> list = List.of(1, 2, 3); list.forEach(i -> System.out.println(i));
```

Q: What is the difference between Function and Predicate?

Function<T,R>: Input = One, Output = Any type, Use Case = Transformations.

Predicate<T>: Input = One, Output = boolean, Use Case = Conditions/filtering.

Q: How does Supplier work?

Supplier<T> takes no argument and returns a value of type T.

```
Supplier<Double> randomSupplier = () -> Math.random();
System.out.println(randomSupplier.get());
```

Q: What is the use of BiFunction?

BiFunction<T, U, R> is used when you need to take two inputs and return one output.

```
BiFunction<Integer, Integer, Integer> adder = (a, b) -> a + b;
System.out.println(adder.apply(10, 20));
```

Q: What is the difference between UnaryOperator and Function?

Function<T,R>: Input T, Return R — General transformation.

UnaryOperator<T>: Input T, Return T — Transformation of same type input/output.

```
UnaryOperator<String> toUpper = s -> s.toUpperCase();
System.out.println(toUpper.apply("java"));
```

Q: How is Stream.map() related to functional interfaces?

map() uses Function<T, R> to transform elements of the stream.

```
List<String> names = List.of("john", "jane"); List<String> upper =
names.stream().map(name -> name.toUpperCase()).toList();
```

Q: What is method reference in Java 8?

Method reference is the replacement of lambda expressions, making code reusable. It is a shorthand notation using :: (double colon) that calls a method by its name instead of writing a full lambda expression.

```
// Lambda messages.forEach(m -> MethodReferenceExample.printMessage(m)); //
Method Reference messages.forEach(MethodReferenceExample::printMessage);
```

Q: What is a Default Method in an interface?

Suppose we have an interface that has a method that doesn't need to be used in all implementing classes — we can put a default method. Advantage: we don't need to forcefully implement it in child classes. Default methods have dummy implementations.

Q: Is it necessary to override default methods?

Default methods have dummy implementations. Implementing classes, if ok with dummy implementations, can use them as-is.

Q: Is the Default keyword an access modifier?

Default is NOT an access modifier like public, private or protected. For default access modifier we cannot use any keywords.

Q: Why were static methods introduced in interfaces in Java 8?

So that you can call those methods with just the interface name — no need to create a class and then an object. Note: Interface never contains Constructors or Static blocks.

Q: What is the difference between map and flatMap?

map(): Follows one-to-one. Input 1 item → Output 1 item. Does not flatten the result. Used for simple transformations.

flatMap(): Follows one-to-many. Input 1 item → Output multiple items. Flattens nested structure into single stream. Used when each element contains a collection inside.

```
// map() List<String> upperNames =
names.stream().map(String::toUpperCase).collect(Collectors.toList()); //
flatMap() List<List<String>> nestedList = List.of(List.of("john","jane"),
List.of("bob","alice")); List<String> flatList =
nestedList.stream().flatMap(Collection::stream).collect(Collectors.toList());
```

Q: What is peek() in Streams?

peek() in Java Streams is used to look at each element in the stream as it passes through, mainly for debugging or logging, without changing the elements.

Q: What are Java 21 features?

Text blocks, Streams, String builder repeat (Sb.repeat("one",5) — repeats 'one' five times), Pattern matching, Some inbuilt math methods (ceilDiv, ceilMod, floorDivExact etc.), Virtual threads, Scoped values.

4. Collections Framework

Q: What is Collection?

It is an interface in java.util package. Represents a group of objects (like List, Set, Queue). Example: List, Set are subtypes of Collection.

Q: What is Collections?

It is a utility class in java.util package. Provides static methods to work with collections (like sort(), shuffle(), reverse()).

Q: Difference between Collection vs Collections?

Collection: Interface — Represents group of objects — Used to define data structures.

Collections: Class — Utility for collection operations — Used to manipulate data structures.

Q: What is the super interface of Collection?

The super interface of Collection in Java is Iterable and the hierarchy is: Iterable → Collection → List/Set/Queue.

Q: What is the difference between HashSet and TreeSet?

HashSet: Uses a Hash Table, No guaranteed order, Allows one null element, When speed is needed.

TreeSet: Uses a Red-Black Tree (Balanced BST), Elements stored in sorted order, Does not allow null, When sorting is needed.

Q: What is the difference between HashMap and TreeMap?

HashMap: No specific order, O(1) for get/put, Allows one null key, Based on hash table, Fast lookups without ordering.

TreeMap: Sorted by keys (natural order or comparator), O(log n) for get/put, Does not allow null keys, Based on Red-Black Tree, Maintain sorted order of keys.

Q: What is the difference between Map and HashMap?

The Map is an interface, and HashMap is a class of the Java collection framework. The Map interface can be implemented by using its implementing classes. The Map contains unique key-pair values. But the HashMap can hold duplicate values.

Q: How does HashMap work internally?

A HashMap stores data as key–value pairs.

1. It uses a hash function on the key to generate a hash code.
2. This hash code is converted into an index in an internal array (called buckets).
3. The value is stored at that index.

Q: Difference between Hashtable vs HashMap?

Hashtable: Does not allow null key or value, Is synchronized, Performance is slow due to synchronization overhead.

HashMap: Allows one null key and multiple null values, Is not synchronized, Faster — not synchronized.

Q: What is the difference between List and LinkedList?

Both are used to store data. List stores data sequentially. LinkedList stores data randomly. LinkedList has many inbuilt features like getFirst(), getLast(), removeFirst().

Q: What is the difference between a shallow copy and a deep copy?

Shallow copy: Creates a new object with the same reference as the original object.

Deep copy: Creates a new object with a new reference.

Q: What is fail-fast and fail-safe iterator?

Fail-Fast Iterators: Detect and fail on concurrent modifications immediately with ConcurrentModificationException.

Fail-Safe Iterators: Work with a snapshot or copy of the collection, allowing modifications without throwing exceptions, but may lead to inconsistent or unpredictable results.

Q: What is the difference between Stack and Queue?

Stack: Follows LIFO (Last In First Out) approach. Example: Undo button — gets the last change we did.

Queue: Follows FIFO (First In First Out) approach. Example: Uploading images and printers.

Q: Java Inbuilt Methods Cheat Sheet

String: length(), charAt(), substring(), equals(), equalsIgnoreCase(), toUpperCase(), toLowerCase(), trim(), split(), replace(), contains()

Math: abs(), max(), min(), sqrt(), pow(), round(), floor(), ceil(), random()

Wrapper: Integer.parseInt(), Double.parseDouble(), valueOf(), compare(), MAX_VALUE, MIN_VALUE

System: System.out.println(), currentTimeMillis(), exit(), gc()

ArrayList: add(), get(), set(), remove(), contains(), size(), clear(), isEmpty()

HashMap: put(), get(), getOrDefault(), remove(), containsKey(), keySet(), values(), entrySet()

Arrays: sort(), binarySearch(), copyOf(), equals(), fill(), toString()

Java 8 Streams: filter(), map(), forEach(), collect(), sorted(), reduce(), count()

Date & Time (Java 8): LocalDate.now(), LocalTime.now(), plusDays(), minusDays(), isBefore(), isAfter()

5. Multithreading & Concurrency

Q: What are the ways to implement multithreading in Java?

There are 2 ways:

1. **Extending the Thread class:** Extend thread class and override run method.
2. **Implementing Runnable Interface:** Create a class that implements Runnable and define the run method.

```
// Option 1 class MyThread extends Thread { public void run() {
System.out.println("Thread running"); } } // Option 2 Runnable r = () ->
System.out.println("Running in thread"); new Thread(r).start();
```

Q: What is the difference between Runnable and Callable?

Runnable: Used when the task does not return a value and does not throw checked exceptions (like a void method).

Callable: Used when the task returns a value and/or throws checked exceptions.

Q: Where can Runnable be used?

At multithreading. Runnable has only one abstract method void run(), so it can be used with lambda expressions and is considered a functional interface.

Q: What is the volatile keyword?

Ensures variable value is always read from main memory rather than thread local memory.

Q: What is the synchronized keyword?

Allows only one thread at a time to access a critical section.

Q: What is the difference between volatile and synchronized?

volatile: Ensures visibility only.

synchronized: Ensures visibility + atomicity using locks.

Q: What is the difference between synchronized method and synchronized block?

Synchronized block: Used to prevent multiple threads from executing a portion of code in a method at the same point in time.

Synchronized method: Will prevent multiple threads from executing the entire method at the same point in time.

Q: What is a Semaphore in multithreading?

A semaphore is a synchronization primitive used to control access to shared resources in a multithreaded environment.

Binary Semaphore (value: 0 or 1): Acts like a mutex/lock. Only one thread can access the resource at a time. Example: protecting a single shared variable.

Counting Semaphore (value: 0 to N): Allows up to N threads concurrently. Example: limiting a connection pool to 10 simultaneous DB connections.

Q: What is the Fork/Join Framework?

A framework used for parallel processing. Tasks are split (forked) and results combined (joined). Introduced in Java 7.

Q: What is ExecutorService for thread management?

Optimize performance with thread pooling for better resource management.

Q: How to call asynchronous methods in multithreading?

Using ExecutorService or CompletableFuture.

In CompletableFuture the methods are:

1. thenApply
2. thenAccept
3. exceptionally

Q: How to handle exceptions while multiple threads are running?

By using exceptionally() in CompletableFuture, we can handle exceptions thrown during async execution.

6. Exception Handling

Q: How do we handle Exceptions?

Exceptions are handled using 3 blocks — try, catch, and finally.

catch: When an exception occurs, this block catches it and works accordingly to handle it or throw it as required.

finally: This block gets executed always regardless of exception occurrence — hence cleanup is done here.

Q: Does finally always execute in Java?

Yes it will, but it will not execute in some cases:

- System.exit() is called.
- When the system crashed.

Q: What is the difference between checked and unchecked exceptions?

Checked exceptions: Checked by the compiler at compile-time and must be explicitly handled or declared in the method signature.

Examples: IOException, SQLException, ClassNotFoundException.

Unchecked exceptions: Not checked by the compiler, do not require explicit handling. Checked at runtime.

Examples: NullPointerException, ArithmeticException, IndexOutOfBoundsException, StackOverflowError, OutOfMemoryError, VirtualMachineError.

Q: How to avoid NullPointerException?

- For collections use Optional.
- For method parameters use @Nonnull annotation.
- Null check before use.

Q: What is Exception Propagation in Java?

Exception propagation refers to the process where an exception is passed from one method to another in the call stack. When an exception occurs in a method, if it's not handled within that method (using a try-catch block), it is automatically propagated to the method that called it, and so on up the call stack.

Q: Difference between ClassNotFoundException and NoClassDefFoundError?

ClassNotFoundException: It is an exception. Occurs when the application tries to load a class at runtime which is not updated in the classpath. Thrown by application itself — via Class.forName(), loadClass(), findSystemClass().

NoClassDefFoundError: It is an error. Occurs when the Java runtime system doesn't find a class definition which is present at compile time but missing at runtime. Thrown by the Java runtime system.

7. Hibernate / JPA

Q: What are Hibernate Cache Levels?

First Level (Session), Second Level (SessionFactory), Query Cache.

Q: What is the Default Cache Time in Hibernate?

No default cache time. Depends on cache provider like Ehcache or Redis.

Q: Difference between get() and load() in Hibernate?

get(): Fetches the data immediately. Returns null if data not found.

load(): Has lazy loading — returns an object that is lazily loaded. Returns an exception if data not found.

Q: What is the merge method in Hibernate?

The merge() method is used to re-attach a detached entity to the persistence context. It copies the state of the given object onto the persistent object with the same identifier and returns the persistent object.

Q: What is the N+1 problem in Hibernate?

The N+1 problem in Hibernate arises from how relationships between entities are handled when lazy loading is used. 1 query loads the list, then N additional queries load each related entity — causing performance issues.

Q: What is the @Embedded annotation?

@Embedded marks a field whose type is annotated with @Embeddable — it embeds the state of the embeddable class directly into the owning entity's table, without a separate table.

Q: Life cycle of JPA entity bean?

Transient: New object created using new. Not saved in database yet.

Persistent: Object is saved using entityManager.persist(). Tracked by JPA — updates go to DB automatically.

Detached: Object was saved before but now disconnected (e.g., after entityManager.close()). Changes are not saved unless re-attached.

Removed: Marked for deletion using entityManager.remove(). Will be deleted from DB on commit.

Q: Explain the flow of JPA entity bean?

• **Entity Class** → You define a POJO annotated with @Entity and usually @Table that maps to a database table.
• **Persistence Context** → When you interact with the database, the EntityManager manages entities inside the persistence context like a cache.
• **Entity Lifecycle:** New (Transient) → Managed (Persistent) → Detached → Removed.
• **Database Sync** → On commit or flush, JPA synchronizes changes from the entity to the actual database (insert, update, delete).

Q: What annotation is used for image and audio file data type?

@Blob annotation — in entity for column we use byte[]. In @Column annotation we give name and columnDefinition as blob. @Blob indicates a large object.

Q: How can we call a stored procedure from the repository?

By using @Query annotation we can call the stored procedure from the repository.

Q: Suppose you are getting DB credential changes every 3 months, how to handle?

There are 2 ways:

1. Updating application.properties file.
2. Using tools like AWS Secrets Manager or Azure Key Vault — these auto-rotate credentials and inject securely into your app.

8. Spring Boot

Q: What is Spring Boot?

Spring Boot is an open-source Java-based framework used to create stand-alone, production-ready Spring applications with minimal configuration. It simplifies the setup and development of new Spring applications by providing default configurations and features like embedded servers (Tomcat, Jetty, etc.), auto-configuration, and a range of built-in tools for rapid development.

Q: What is the Spring Boot entry point?

Class annotated with @SpringBootApplication having the main method.

Q: What are the main features of Spring Boot?

• **Auto-Configuration:** Automatically configures Spring application based on the project's dependencies.
• **Spring Boot Starters:** A set of dependency descriptors to include necessary libraries.
• **Embedded Servers:** Provides built-in servers like Tomcat or Jetty.
• **Spring Boot CLI:** A command-line tool for quickly prototyping Spring applications.
• **Actuator:** Provides production-ready features like monitoring and managing applications.
• **Standalone Applications:** Allows creation of Java applications that can run with java -jar.

Q: What is the difference between Spring and Spring Boot?

Spring: A comprehensive framework for building Java applications. Requires extensive configuration.

Spring Boot: A sub-project of Spring that simplifies application development by offering pre-configured setups and defaults, reducing boilerplate code. Ideal for microservices.

Q: What is a Spring Boot Starter?

Spring Boot Starters are a set of convenient dependency descriptors that aggregate commonly used libraries for specific functionalities.

- spring-boot-starter-web: Includes dependencies for building web applications.
- spring-boot-starter-data-jpa: Includes dependencies for JPA and Spring Data.
- spring-boot-starter-security: Includes dependencies for Spring Security.

Q: What does @SpringBootApplication annotation do internally?

@SpringBootApplication is equivalent to three annotations:

- **@Configuration:** Helps in identifying beans for a particular Spring dependency injection.
- **@EnableAutoConfiguration:** Helps you to auto scan — scans for any bean and adds those dependencies.
- **@ComponentScan:** Used to scan components and beans.

Q: What is @Bean and @Configuration?

@Bean: Used to explicitly define a bean in a Spring configuration class. The method annotated with @Bean returns an object that will be registered and managed by the Spring container.

@Configuration: Used to mark a class as a source of bean definitions. It tells Spring that this class contains one or more @Bean methods.

Q: What is Spring Boot Auto-Configuration?

Spring Boot's auto-configuration feature automatically configures your Spring application based on the dependencies present in the classpath. For example, if spring-boot-starter-web is in the classpath, it automatically configures Spring MVC, a dispatcher servlet, and a default embedded server (Tomcat).

Q: What is the role of application.properties file in Spring Boot?

Used to define application-level configurations:

- Server properties: server.port=8081
- Database configurations: spring.datasource.url=jdbc:mysql://localhost:3306/mydb
- Logging levels: logging.level.org.springframework=DEBUG
- Custom properties: myapp.custom.property=value

Q: What is Spring Boot Actuator?

Spring Boot Actuator provides production-ready features to monitor and manage your application. It exposes endpoints such as:

- /actuator/health: Displays health status.
- /actuator/metrics: Memory usage, CPU usage, and other stats.

• /actuator/env: Displays environment properties.

Integrates with monitoring tools like Prometheus and Grafana.

```
<!-- pom.xml --> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId> <version>3.4.0</version>
</dependency> # application.properties management.endpoints.web.base=/manage #
Postman: http://localhost:8080/manage
```

Q: What is an embedded server in Spring Boot?

A server packaged with your application, allowing it to run as a stand-alone Java application without needing to deploy WAR files to an external server. Spring Boot supports Tomcat, Jetty, and Undertow.

Q: Can we use a separate server instead of Tomcat?

Yes. Remove embedded Tomcat server, change packaging to WAR, deploy to external server. Or use spring-boot-starter-jetty as a dependency.

Q: Is it possible to change the port of the embedded Tomcat server?

Yes. Set server.port=8077 in application.properties.

Q: What is Spring Boot DevTools?

Key features:

1. **Automatic Restart:** Automatically restarts application when any code changes are made.
2. **Live Reload Integration:** Refreshes the browser automatically when HTML, CSS, or JavaScript changes are detected.
3. **Remote Debugging:** Allows connecting to a remotely running application and automatically refreshes it on changes.

Q: How does Spring Boot handle security?

Spring Boot provides security out-of-the-box through Spring Security. By adding spring-boot-starter-security, you automatically enable basic authentication. Customize using @EnableWebSecurity and configure rules through WebSecurityConfigurerAdapter.

Q: What is the difference between @Controller and @RestController?

@Controller: Used to define a controller that handles web requests and typically returns a view (e.g., HTML). Without @ResponseBody, you get a white label error when calling the endpoint.

@RestController: Combines @Controller + @ResponseBody. Every method returns data (JSON/XML) directly. Instead of writing @ResponseBody on each and every API call, we can use @RestController.

Q: What is the difference between @RequestMapping and @GetMapping?

@RequestMapping will accept all types of calls like GET, POST, PUT. @GetMapping is the extension of @RequestMapping — only accepts GET requests.

Q: What are stereotype annotations in Spring Boot?

@Service, @Repository, @Controller are the stereotype annotations in Spring Boot.

Q: What are Spring autowiring types?

1. Constructor
2. @Autowired
3. byName
4. byType
5. Default

Q: Explain the Spring Bean Lifecycle.

1. **Instantiation:** The bean is created using its default constructor or a factory method.
2. **Dependency Injection:** Dependencies are injected based on configuration (XML, annotations, or Java-based).
3. **Post-Initialization:** Custom initialization methods specified in the configuration are executed.
4. **Bean Ready for Use:** The bean is fully initialized and ready to be used.
5. **Pre-Destruction:** Custom destroy methods specified in the configuration are executed.
6. **Destruction:** The bean is removed from the container and resources are released.

Q: What is a Spring Boot Profile?

Profiles in Spring Boot allow you to create different configurations for different environments (DEV, QA, PROD). Define environment-specific properties in files like application-dev.properties, application-test.properties. Activate via spring.profiles.active in application.properties.

Q: What is @ConfigurationProperties in Spring Boot?

Used to bind external configurations from properties or YAML files to a Java object. Map properties with a common prefix (e.g., app.datasource) to a POJO by annotating the class with @ConfigurationProperties(prefix="app.datasource").

Q: How to disable a specific auto-configuration class?

Use the exclude attribute of @EnableAutoConfiguration.

```
@EnableAutoConfiguration(exclude = {DataSourceAutoConfiguration.class})
```

Q: How do you define a custom exception in Spring Boot?

1. Create a custom exception class: public class ResourceNotFoundException extends RuntimeException {}.
2. Use @ResponseStatus to associate an HTTP status code: @ResponseStatus(HttpStatus.NOT_FOUND).
3. Create a global exception handler using @ControllerAdvice and handle using @ExceptionHandler.

Q: How do you handle exceptions in Spring Boot?

• **Global Exception Handling:** Use @ControllerAdvice and @ExceptionHandler to catch and handle exceptions across the entire application.
• **Custom Error Pages:** Define custom error pages by creating ErrorController or configuring error views in application.properties.
• **ResponseEntityExceptionHandler:** Extend this class to create a centralized exception handling mechanism with specific HTTP status codes.

Q: What is @Transactional annotation?

@Transactional annotation is used to execute the entire task as one unit. For example, if we need to save data in both Employee and Address table, but only Employee data is saved and Address data fails — @Transactional will not save any data and will roll back that issue, maintaining data consistency.

Q: What is OAuth?

OAuth is an authorization framework. Web services are secured using auth tokens (JWT). JWT Token Params: sub (user ID), iat (issued at), exp (expiration), iss (issuer), aud (audience), role/scope. OAuth Token Request Params: grant_type, client_id, client_secret, username, password, scope, redirect_uri, code.

Q: What is an Application Server?

Provides web container, transactions, security, and business logic support. Types: JBoss, WebLogic, WebSphere, GlassFish. Tomcat is only a web server.

Q: How to deploy a Spring Boot web application as JAR and WAR?

Add the Spring Boot Maven plugin in pom.xml and set packaging:

```
<plugin> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId> </plugin>
<packaging>jar</packaging> <!-- or --> <packaging>war</packaging>
```

9. Java Version Comparison (8 / 17 / 21)

Q: Java 8 vs Java 17 vs Java 21 — Key Differences

Java 8 (March 2014 — LTS): Lambda expressions, Default methods, Stream API, Optional Class, Date and Time API.

Java 17 (September 2021 — LTS): Sealed classes, Switch expressions, Text blocks, Pattern matching. G1 GC default, ZGC. Removed Nashorn and RMI activation.

Java 21 (September 2023 — LTS): Virtual threads, Scoped values (JEP 446), String templates (preview), Enhanced pattern matching, Sequenced Collections. Refined ZGC, Shenandoah GC. StringBuilder.repeat().

Q: What are the Java 21 notable additions?

Text blocks, Streams, String builder repeat (Sb.repeat("one",5)), Pattern matching using triple quotes, Scoped values as a better alternative to thread-local variables for immutable data sharing, some inbuilt Math methods (ceilDiv, ceilMod, floorDivExact, divideExact, ceilDivExact).

10. Experience & Scenario-Based Questions

Q: How did you optimize performance of a Java application?

- Used profiling tools (JVisualVM, JProfiler) to identify bottlenecks.
- Optimized database queries with indexes and batch processing.
- Reduced object creation, used caching (Ehcache/Redis).
- Applied asynchronous processing where needed.

Q: How did you refactor legacy Java code?

- Added unit tests before changes.
- Broke down large classes/methods into smaller ones.
- Replaced deprecated APIs with modern ones.
- Applied design patterns for better maintainability.

Q: How did you ensure the team adheres to Java coding best practices?

- Enforced code reviews and static code analysis (SonarQube, Checkstyle).
- Followed coding standards (naming, exception handling, logging).
- Conducted knowledge-sharing sessions.

Q: Explain scenarios where you implemented multithreading and addressed concurrency issues?

- Used multithreading for parallel data processing and background tasks.
- Handled concurrency with synchronized, Locks, and ConcurrentHashMap.
- Used ExecutorService instead of manual thread creation.
- Avoided deadlocks using proper lock ordering.

Q: How do you handle load balancing at client and server level?

- **Client level:** Implemented retry logic and round-robin requests.
- **Server level:** Used load balancers like Nginx/HAProxy or cloud-based (AWS ELB).
- Scaled horizontally using microservices.

Q: In your Spring application if you get Out of Memory issue how to handle?

- Analyzed heap dump with tools (MAT, VisualVM).
- Fixed memory leaks (closing connections, removing unused objects).
- Used caching wisely with eviction policies.
- Increased JVM heap size with -Xmx.

Q: What is a memory leak in Java?

A memory leak in Java happens when objects are no longer needed but are still referenced, so the Garbage Collector cannot remove them.

Q: What are some common performance optimization techniques in Java?

• **Efficient Data Structures:** Use appropriate collections like ArrayList, HashMap, or ConcurrentHashMap based on use cases.
• **Avoid Unnecessary Object Creation:** Reuse objects and avoid frequent instantiations.
• **String Handling:** Use StringBuilder for string manipulations instead of concatenation with +.
• **Lazy Initialization:** Initialize objects only when needed.
• **Memory Management:** Monitor and optimize garbage collection.
• **Streams and Parallelism:** Use Java 8 Streams carefully; prefer parallel streams for large, independent tasks.
• **Caching:** Implement caching mechanisms for frequently used data.
• **Database Optimization:** Optimize SQL queries and use connection pooling.
• **Concurrency:** Use efficient threading and avoid synchronization overhead.

---

11. Additional Core Java Topics (4+ Years)

Q: What is the Factory Method pattern? Give an example.

Factory Method defines an interface for creating an object but lets subclasses decide which class to instantiate. It removes the object creation logic from the client.

```
// Shape interface interface Shape { void draw(); } // Concrete classes class Circle implements Shape { public void draw() { System.out.println("Drawing Circle"); } } class Rectangle implements Shape { public void draw() { System.out.println("Drawing Rectangle"); } } // Factory class class ShapeFactory { public static Shape getShape(String type) { if (type.equals("CIRCLE")) return new Circle(); if (type.equals("RECTANGLE")) return new Rectangle(); return null; } } // Usage Shape shape = ShapeFactory.getShape("CIRCLE"); shape.draw(); // Drawing Circle
```

Q: What is the Builder pattern? Give an example.

Builder pattern constructs complex objects step by step. It separates the construction of an object from its representation. Very useful when an object has many optional fields.

```
class Person { private String name; // required private int age; // required private String email; // optional private String phone; // optional private Person(Builder builder) { this.name = builder.name; this.age = builder.age; this.email = builder.email; this.phone = builder.phone; } public static class Builder { private String name; private int age; private String email; private String phone; public Builder(String name, int age) { this.name = name; this.age = age; } public Builder email(String email) { this.email = email; return this; } public Builder phone(String phone) { this.phone = phone; return this; } public Person build() { return new Person(this); } } } // Usage Person p = new Person.Builder("John", 25) .email("john@gmail.com") .phone("9999999999") .build();
```

Q: What is the Strategy pattern? Give an example.

Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Lets the algorithm vary independently from clients that use it.

```
// Strategy interface interface SortStrategy { void sort(int[] arr); } // Concrete strategies class BubbleSort implements SortStrategy { public void sort(int[] arr) { System.out.println("Bubble Sort"); } } class QuickSort implements SortStrategy { public void sort(int[] arr) { System.out.println("Quick Sort"); } } // Context class Sorter { private SortStrategy strategy; public Sorter(SortStrategy strategy) { this.strategy = strategy; } public void sort(int[] arr) { strategy.sort(arr); } } // Usage Sorter sorter = new Sorter(new QuickSort()); sorter.sort(new int[]{5, 3, 1});
```

Q: What is the Decorator pattern? Give an example.

Decorator pattern adds new behavior to objects dynamically by wrapping them. Used for adding features like logging, caching without modifying the original class.

```
interface Logger { void log(String message); } class ConsoleLogger implements Logger { public void log(String message) { System.out.println(message); } } // Decorator class TimestampLogger implements Logger { private Logger logger; public TimestampLogger(Logger logger) { this.logger = logger; } public void log(String message) { logger.log("[" + System.currentTimeMillis() + "] " + message); } } // Usage Logger logger = new TimestampLogger(new ConsoleLogger()); logger.log("User logged in");
```

Q: What is the difference between final, finally, and finalize?

final: Keyword used to restrict modification.

- final variable = constant (value cannot be changed).
- final method = cannot be overridden.
- final class = cannot be extended.

finally: Block in exception handling that always executes regardless of whether an exception occurred or not. Used for cleanup (closing connections, streams).

finalize(): Method called by the Garbage Collector just before an object is destroyed. Used to release resources. Deprecated since Java 9.

```
// final final int MAX = 100; // cannot reassign // finally try { int result = 10 / 0; } catch (Exception e) { System.out.println("Exception caught"); } finally { System.out.println("Always runs"); } // finalize @Override protected void finalize() throws Throwable { System.out.println("Object destroyed"); }
```

Q: How does the String Pool work internally?

The String Pool (also called String Intern Pool) is a special memory area inside the Heap where Java stores String literals.

- When you write String s = "Hello", JVM checks if "Hello" already exists in the pool.
- If yes — it returns the same reference. If no — it creates a new String in the pool.

• When you use new String("Hello"), it always creates a new object in the heap — NOT in the pool.

• Use .intern() to manually add a String to the pool.

```
String s1 = "Hello"; // goes to String Pool String s2 = "Hello"; // reuses same pool object String s3 = new String("Hello"); // creates new heap object System.out.println(s1 == s2); // true (same pool reference) System.out.println(s1 == s3); // false (different objects) System.out.println(s1.equals(s3)); // true (same value)
```

Q: What are Generics in Java?

Generics allow you to write classes, interfaces, and methods that work with any data type while providing compile-time type safety. They eliminate the need for casting and prevent ClassCastException at runtime.

```
// Generic class class Box<T> { private T value; public void set(T value) { this.value = value; } public T get() { return value; } } // Usage Box<Integer> intBox = new Box<>(); intBox.set(10); int val = intBox.get(); // no casting needed // Generic method public <T> void print(T item) { System.out.println(item); } // Bounded type - only Number and subclasses public <T extends Number> double sum(T a, T b) { return a.doubleValue() + b.doubleValue(); } // Wildcard public void printList(List<?> list) { for (Object obj : list) System.out.println(obj); }
```

Q: What are Enums and their benefits? Deep usage.

Enum is a special class in Java that represents a group of constants. Benefits: type safety, readability, works in switch statements, can have methods and fields.

```
public enum Day { MONDAY("Start of work week"), FRIDAY("End of work week"), SUNDAY("Rest day"); private String description; Day(String description) { this.description = description; } public String getDescription() { return description; } } // Usage Day day = Day.MONDAY; System.out.println(day.getDescription()); // Start of work week // In switch switch (day) { case MONDAY: System.out.println("Go to work"); break; case SUNDAY: System.out.println("Rest"); break; }
```

Q: What are the types of Inner Classes in Java?

1. Static Nested Class: Declared static inside a class. Can be accessed without creating outer class instance.

2. Non-static Inner Class: Has access to outer class members. Needs outer class instance to create.

3. Local Inner Class: Defined inside a method. Only accessible within that method.

4. Anonymous Inner Class: A class without a name, declared and instantiated at the same time. Commonly used for implementing interfaces inline.

```
// Static Nested class Outer { static class StaticNested { void show() { System.out.println("Static Nested"); } } } Outer.StaticNested obj = new Outer.StaticNested(); // Anonymous Inner Class Runnable r = new Runnable() { public void run() { System.out.println("Running"); } }; new Thread(r).start();
```

Q: What are the access modifiers in Java and their scope?

public: Accessible from anywhere — same class, same package, subclass, other packages.

protected: Accessible within same package and subclasses (even in different packages).

default (no keyword): Accessible only within the same package. Not accessible outside the package.

private: Accessible only within the same class. Most restrictive.

```
public class MyClass { public int a = 1; // accessible everywhere protected int b = 2; // accessible in package + subclasses int c = 3; // default - package only private int d = 4; // this class only }
```

Q: What is Autoboxing and Unboxing? What are the pitfalls?

Autoboxing: Automatic conversion of primitive to its wrapper class (int → Integer).

Unboxing: Automatic conversion of wrapper class to primitive (Integer → int).

Pitfalls:

• NullPointerException when unboxing a null wrapper object.

• Performance overhead — unnecessary object creation in loops.

• == comparison with Integer cache: Integer cache only works for values -128 to 127.

```
// Autoboxing int a = 5; Integer boxed = a; // auto-boxed // Unboxing Integer x = null; int y = x; // NullPointerException! // Integer cache trap Integer i1 = 127; Integer i2 = 127; System.out.println(i1 == i2); // true (cached) Integer i3 = 128; Integer i4 = 128; System.out.println(i3 == i4); // false (not cached) System.out.println(i3.equals(i4)); // true
```

Q: What is the var keyword in Java (Java 10+)?

var is a local variable type inference keyword introduced in Java 10. The compiler automatically infers the type from the right-hand side. It reduces boilerplate but the type is still fixed at compile time — it is NOT dynamic like JavaScript.

Restrictions: Cannot be used for method parameters, return types, or class fields. Cannot be initialized with null.

```
// Without var ArrayList<String> list = new ArrayList<String>(); // With var var list = new ArrayList<String>(); // type inferred var name = "John"; // String var age = 25; // int var map = new HashMap<String, Integer>(); // NOT allowed var x = null; // error - cannot infer type public var name; // error - not for fields
```

Q: What is instanceof and Pattern Matching (Java 16+)?

instanceof checks whether an object is an instance of a specific class. Java 16 introduced pattern matching for instanceof — it combines type check and cast in one step.

```
// Old way Object obj = "Hello"; if (obj instanceof String) { String s = (String) obj; // explicit cast System.out.println(s.length()); } // New way - Pattern Matching (Java 16+) if (obj instanceof String s) { // check + cast in one line System.out.println(s.length()); }
```

Q: How did Java 8 change the rules of Abstract Class vs Interface?

Before Java 8, interfaces could only have abstract methods. Java 8 added default and static methods to interfaces, which blurred the line between abstract classes and interfaces.

Still different:

• Abstract class can have constructors, instance variables, and state. Interface cannot.

• A class can implement multiple interfaces but extend only one abstract class.

• Abstract class can have any access modifier on methods. Interface methods are public by default.

• Use abstract class when classes share common state. Use interface for defining a contract/capability.

```
// Interface with default method (Java 8) interface Vehicle { void start(); // abstract default void stop() { // default - no need to override System.out.println("Vehicle stopped"); } static void info() { // static System.out.println("Vehicle interface"); } } // Abstract class with state abstract class Animal { private String name; // state - not possible in interface public Animal(String name) { this.name = name; } abstract void sound(); public String getName() { return name; } }
```

Q: What is a Deadlock in Java? How to prevent it?

A deadlock occurs when two or more threads are blocked forever, each waiting for the other to release a lock.

How to prevent:

• Always acquire locks in the same order across all threads.

• Use tryLock() with timeout instead of synchronized.

• Avoid nested locks where possible.

• Use higher-level concurrency tools like ExecutorService, ConcurrentHashMap.

```
// Deadlock example Object lock1 = new Object(); Object lock2 = new Object(); // Thread 1: acquires lock1, then tries lock2 // Thread 2: acquires lock2, then tries lock1 // Both wait forever = DEADLOCK // Prevention - always lock in same order synchronized (lock1) { synchronized (lock2) { // always lock1 first, then lock2 } }
```

Q: How does ConcurrentHashMap work internally?

ConcurrentHashMap allows concurrent read and write operations without locking the entire map.

• In Java 7: Used segment-level locking — divided map into 16 segments, each with its own lock.

• In Java 8+: Uses bucket-level locking with CAS (Compare-And-Swap) operations. Only the specific bucket being modified is locked, not the entire map.

• Reads are non-blocking — no lock needed for get().

• Does NOT allow null keys or null values (unlike HashMap).

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>(); map.put("a", 1); map.put("b", 2); // Thread-safe operations map.putIfAbsent("c", 3); map.computeIfAbsent("d", k -> k.length()); // NOT allowed map.put(null, 1); // NullPointerException map.put("a", null); // NullPointerException
```

Q: What is Optional in Java 8? How to use it?

Optional is a container class that may or may not contain a non-null value. It is used to avoid NullPointerException and make null handling explicit.

```
// Creating Optional Optional<String> opt1 = Optional.of("Hello"); // non-null value Optional<String> opt2 = Optional.empty(); // empty Optional<String> opt3 = Optional.ofNullable(null); // may be null // Checking and getting if (opt1.isPresent()) { System.out.println(opt1.get()); } // orElse - default value if empty String result = opt3.orElse("Default"); // orElseThrow String val = opt3.orElseThrow(() -> new RuntimeException("Not found")); // ifPresent opt1.ifPresent(s -> System.out.println(s.toUpperCase())); // map + filter Optional<Integer> len = opt1.map(String::length).filter(l -> l > 3);
```

Q: How to sort a list of objects using Comparable and Comparator?

```
class Employee implements Comparable<Employee> { String name; int age; Employee(String name, int age) { this.name=name; this.age=age; } // Natural ordering by age @Override public int compareTo(Employee other) { return this.age - other.age; } } // Using Comparable List<Employee> list = Arrays.asList( new Employee("John", 30), new Employee("Alice", 25) ); Collections.sort(list); // sorts by age // Using Comparator (custom - sort by name) list.sort(Comparator.comparing(e -> e.name)); // Multiple fields list.sort(Comparator.comparing((Employee e) -> e.age) .thenComparing(e -> e.name));
```

Q: What is the Exception hierarchy in Java?

Throwable is the root class of all errors and exceptions in Java.

Throwable

■■■ **Error** — serious problems, not meant to be caught (OutOfMemoryError, StackOverflowError)

■■■ **Exception**

■■■ **RuntimeException (Unchecked)** — NullPointerException, ArrayIndexOutOfBoundsException, ClassCastException

■■■ **Checked Exceptions** — IOException, SQLException, ClassNotFoundException

```
// Custom checked exception class InsufficientFundsException extends Exception { public InsufficientFundsException(String message) { super(message); } } // Custom unchecked exception class InvalidAgeException extends RuntimeException { public InvalidAgeException(String message) { super(message); } } // throw vs throws public void withdraw(double amount) throws InsufficientFundsException { if (amount > balance) { throw new InsufficientFundsException("Not enough balance"); } }
```

Q: Write a complete Immutable class in Java.

```
import java.util.Collections; import java.util.List; public final class ImmutableStudent { // final - cannot extend private final String name; // final fields private final int age; private final List<String> courses; // mutable field - need defensive copy public ImmutableStudent(String name, int age, List<String> courses) { this.name = name; this.age = age; // defensive copy of mutable field this.courses = Collections.unmodifiableList(courses); } // Only getters - no setters public String getName() { return name; } public int getAge() { return age; } public List<String> getCourses() { return courses; } }
```

Q: What is try-with-resources in Java?

try-with-resources (Java 7+) automatically closes resources (like files, connections) after the try block finishes — even if an exception occurs. The resource must implement AutoCloseable or Closeable interface.

```
// Without try-with-resources (old way) BufferedReader br = null; try { br = new BufferedReader(new FileReader("file.txt")); String line = br.readLine(); } catch (IOException e) { e.printStackTrace(); } finally { if (br != null) br.close(); // manual close } // With try-with-resources (new way) try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) { String line = br.readLine(); } catch (IOException e) { e.printStackTrace(); } // br automatically closed here
```

Q: Important String methods asked in interviews with examples.

```
String s = "Hello World"; s.length(); // 11 s.charAt(0); // 'H' s.indexOf("World"); // 6 s.substring(6); // "World" s.substring(0, 5); // "Hello" s.toLowerCase(); // "hello world" s.toUpperCase(); // "HELLO WORLD" s.trim(); // removes leading/trailing spaces s.replace("World","Java"); // "Hello Java" s.contains("Hello"); // true s.startsWith("Hello"); // true s.endsWith("World"); // true s.split(" "); // ["Hello", "World"] s.equals("Hello World"); // true s.equalsIgnoreCase("hello world"); // true String.valueOf(123); // "123" Integer.parseInt("123"); // 123
```